MODULE 19

Simple RNN — In-Depth Intuition

Lectures Covered

Lecture 1 — Why RNNs are Needed
Lecture 2 — Problem with ANN for Sequential Data
Lecture 3 — What is a Recurrent Neural Network
Lecture 4 — Forward Propagation in RNN with Architecture
Lecture 5 — Types of RNN — Many To One
Lecture 6 — Other Types of RNN for Gen AI Applications
Lecture 7 — Backpropagation in RNN
Lecture 8 — Backpropagating Through Time (BPTT)
Lecture 9 — Problems with RNN
Lecture 10 — Vanishing Gradient Problem in Long-Term RNNs

Lecture 1: Why RNNs are Needed

Standard feedforward networks (ANNs, CNNs) treat each input independently — they have no memory of previous inputs. But many real-world problems involve sequences where context and order matter enormously. Recurrent Neural Networks (RNNs) were designed specifically to handle this class of problems.

Sequential Data — What Makes It Different?

In sequential data, the meaning of an element depends on what came before it. Consider:

- **Text:** "The bank can guarantee deposits will eventually cover future tuition costs." — "bank" = financial OR riverbank?
- **Speech:** The same word sounds different depending on the phoneme preceding it
- **Stock prices:** Today's price depends on the entire preceding history of prices
- **Video:** Each frame only makes sense given the frames before it
- **Music:** A note's emotional effect depends on the notes that led to it

Real-World Applications That Need RNNs

Application	Why Sequential Context is Needed
Machine Translation	Word order and grammatical structure differ between languages; entire sentence context required
Speech Recognition	Phonemes blend together; disambiguation needs surrounding audio context
Text Generation / LLMs	Each generated token depends on all previous tokens in the sequence
Sentiment Analysis	Negation ('not good') flips meaning — order and context are essential
Time Series Forecasting	Future value depends on patterns in historical data over time
Music Generation	Rhythm, harmony, and melody are defined by temporal relationships
Video Captioning	Caption must describe motion and action across multiple frames
Named Entity Recognition	Knowing whether 'Paris' is a city or a name requires surrounding words

The core insight

A feedforward network sees one snapshot. An RNN sees a movie.
The 'recurrent' connection passes a hidden state forward in time, giving the network a form of memory.
This is what allows RNNs to model temporal dependencies and long-range context.

Lecture 2: Problem with ANN for Sequential Data

To understand why RNNs exist, we must first understand precisely why vanilla ANNs fail on sequential problems. There are three fundamental structural incompatibilities.

Problem 1 — Fixed Input Size

A standard ANN has a fixed input layer. If you want to process a sentence of 5 words and a sentence of 50 words, you cannot use the same network without padding or truncation. Padding wastes computation; truncation loses information.

```
# ANN approach — must fix input length
model = Sequential([
    Dense(128, input_shape=(MAX_SEQUENCE_LENGTH,)),  # hard-coded!
    Dense(64, activation='relu'),
    Dense(num_classes, activation='softmax')
])

# Problem: sentence 'Hi' and sentence 'The quick brown fox jumps over...'
# Both must be padded/truncated to MAX_SEQUENCE_LENGTH
# Short sequences waste 95% of their input slots
# Long sequences lose their tail — information destroyed
```

Problem 2 — No Parameter Sharing Across Time

If you encode a sequence by unrolling it into one long vector and feeding it to an ANN, each position in the sequence has its own set of weights. The word 'cat' at position 3 and 'cat' at position 15 are handled by entirely different neurons — the network cannot generalise.

Property	ANN Behaviour | Desired Behaviour
Weight reuse	Different weights for position 1, 2, 3... N | Same weights applied at every time step
Learning generalisation	Must re-learn that 'not' negates meaning at every position | Learn once, apply everywhere
Parameter count	Grows linearly with sequence length | Fixed regardless of sequence length

Problem 3 — No Memory / No Temporal Context

An ANN has no mechanism to remember what it saw two steps ago. When predicting the last word in 'The clouds in the sky are ___', the ANN processes each word independently — it cannot connect 'clouds' and 'sky' to arrive at 'blue'.

The fundamental mismatch

ANN: maps input X directly to output Y. No internal state.
Sequential task: output at time t depends on inputs at times 1, 2, ..., t.
Feeding a flattened sequence to an ANN destroys all temporal ordering information.
The network has no idea which tokens were adjacent, which came first, or which came last.

Summary of ANN Failures on Sequential Data

Limitation	Consequence
Fixed input size	Cannot handle variable-length sequences naturally
No parameter sharing	Model must re-learn patterns at every sequence position
No temporal memory	Cannot model long-range dependencies between tokens

Limitation	Consequence
Positional blindness	Swapping 'dog bites man' and 'man bites dog' produces same output
Scalability	A sequence of length 1000 requires 1000x more input neurons

Lecture 3: What is a Recurrent Neural Network?

A Recurrent Neural Network (RNN) is a neural network with a feedback loop — the output (hidden state) at each time step is fed back as an additional input at the next time step. This creates a form of memory that persists across the sequence.

The Key Idea — Hidden State as Memory

At each time step t, the RNN computes a hidden state h_t from two sources:

- **Current input** x_t — the token/feature at this time step
- **Previous hidden state** h_(t-1) — the network's memory of everything it has seen so far
- **Output** y_t — optionally produced at each step, or only at the final step

```
h_t  =  f( W_hh * h_(t-1)  +  W_xh * x_t  +  b_h )

y_t  =  g( W_hy * h_t  +  b_y )

where  f = tanh (typically),  g = softmax or linear
```

Key Components

Symbol	Meaning | Role
x_t	Input vector at time step t | Current token embedding or feature vector
h_t	Hidden state at time t | The 'memory' — encodes everything seen up to step t
h_(t-1)	Previous hidden state | Passed forward from last step — carries temporal context
y_t	Output at time t | Prediction, classification, or next-token probability
W_xh	Input-to-hidden weight matrix | Shared across ALL time steps (parameter sharing!)
W_hh	Hidden-to-hidden weight matrix | The recurrent weight — what gives RNNs their memory
W_hy	Hidden-to-output weight matrix | Maps memory to prediction
b_h, b_y	Bias vectors | Standard bias terms

Unrolling an RNN Through Time

An RNN with a feedback loop can be 'unrolled' into a deep feedforward network — one copy per time step, all sharing the same weights. This is the key to understanding both training (BPTT) and why deep dependencies are hard to learn.

Unrolling intuition

Step 1: h_1 = f(W_xh * x_1 + W_hh * h_0 + b_h)
Step 2: h_2 = f(W_xh * x_2 + W_hh * h_1 + b_h)
Step 3: h_3 = f(W_xh * x_3 + W_hh * h_2 + b_h)
Step T: h_T = f(W_xh * x_T + W_hh * h_(T-1) + b_h)

Same W_xh and W_hh at every step — parameter sharing across time!

RNN vs ANN — Direct Comparison

Property	ANN | RNN
Input handling	Fixed-size vector | Variable-length sequence
Memory	None — stateless | Hidden state h_t carries context
Parameter sharing	Different weights per layer | Same weights reused at every time step
Temporal order	Ignored | Explicitly modelled via recurrent connection
Computation graph	Acyclic (DAG) | Cyclic — unrolled into deep graph for training

```
# Simple RNN in Keras
from tensorflow.keras.layers import SimpleRNN, Dense, Embedding
from tensorflow.keras.models import Sequential

model = Sequential([
    Embedding(vocab_size, 64),       # token -> dense vector
    SimpleRNN(128, return_sequences=False),  # RNN layer
    Dense(num_classes, activation='softmax') # classification head
])

# return_sequences=True  -> output h_t at every step (for seq2seq)
# return_sequences=False -> output only final h_T (for classification)
```

Lecture 4: Forward Propagation in RNN with Architecture

Forward propagation in an RNN processes the input sequence one token at a time, updating the hidden state at each step. Understanding the exact matrix operations and dimensions is essential before tackling training.

Architectural Dimensions

Variable	Dimension | Meaning
x_t	(batch, input_size) | One time step of input features
h_t	(batch, hidden_size) | Hidden state — the memory vector
W_xh	(input_size, hidden_size) | Maps input to hidden space

Variable	Dimension | Meaning
W_hh	(hidden_size, hidden_size) | Recurrent weight — maps prev hidden to current
W_hy	(hidden_size, output_size) | Maps hidden state to output
b_h	(hidden_size,) | Hidden bias
b_y	(output_size,) | Output bias

Step-by-Step Forward Pass

Step 1 — Initialise Hidden State

Before processing the first token, initialise h_0 (typically to zeros):

```
h_0  =  zeros( hidden_size )
```

Step 2 — Process Each Token

For each time step t = 1 to T:

```
a_t  =  W_xh * x_t  +  W_hh * h_(t-1)  +  b_h

h_t  =  tanh( a_t )

y_t  =  softmax( W_hy * h_t  +  b_y )    (if output needed at step t)
```

Step 3 — Collect Output

- Many-to-one: use only y_T (final hidden state) for prediction
- Many-to-many: use y_1, y_2, ..., y_T (output at every step)

Why tanh as Activation?

Property	tanh | Why It Matters for RNNs
Output range	(-1, +1) | Keeps hidden state values bounded — prevents exploding state
Zero-centred	Yes | Positive and negative updates balanced — faster learning
Gradient	1 - tanh^2(x) | Gradient near 0 for large inputs — vanishing gradient risk
vs ReLU	Bounded | ReLU unbounded — can cause exploding hidden state in RNNs

Concrete Numerical Example

```
import numpy as np

# Hyperparameters
input_size  = 4   # e.g. one-hot encoded vocabulary of 4
hidden_size = 3
```

```
output_size = 4   # predicting next character

# Initialise weights (random small values in practice)
W_xh = np.random.randn(input_size,  hidden_size) * 0.01
W_hh = np.random.randn(hidden_size, hidden_size) * 0.01
W_hy = np.random.randn(hidden_size, output_size) * 0.01
b_h  = np.zeros(hidden_size)
b_y  = np.zeros(output_size)

def softmax(x):
    e = np.exp(x - np.max(x))
    return e / e.sum()

# Forward pass for a sequence ['h','e','l','l']
sequence = [np.array([1,0,0,0]),   # 'h'
            np.array([0,1,0,0]),   # 'e'
            np.array([0,0,1,0]),   # 'l'
            np.array([0,0,1,0])]   # 'l'

h = np.zeros(hidden_size)   # h_0
for t, x in enumerate(sequence):
    a = x @ W_xh + h @ W_hh + b_h   # pre-activation
    h = np.tanh(a)                   # hidden state
    y = softmax(h @ W_hy + b_y)      # output distribution
    print(f't={t+1}  h={h.round(3)}  y={y.round(3)}')
```

Lecture 5: Types of RNN — Many To One

Not all sequence problems have the same input-output structure. The 'Many-to-One' architecture is one of the most commonly used RNN configurations, and serves as the foundation for understanding all other types.

The Many-to-One Architecture

In Many-to-One RNNs, the model receives a full sequence of inputs but produces only a single output at the end. The final hidden state h_T acts as a compressed summary of the entire input sequence.

```
Input:   x_1, x_2, x_3, ..., x_T    (sequence)
Hidden:  h_1, h_2, h_3, ..., h_T    (updated at each step)
Output:  y_T only                   (single prediction from h_T)
```

Applications of Many-to-One

Application	Input Sequence | Single Output
Sentiment Analysis	Sequence of word embeddings | Positive / Negative / Neutral label
Document Classification	Sequence of sentence vectors | Category / Topic label
Spam Detection	Sequence of email tokens | Spam / Not Spam
Music Genre Classification	Sequence of audio frames | Genre label
Stock Trend Prediction	Sequence of daily OHLCV data | Up / Down / Neutral
Intent Detection	User utterance tokens | Intent class (book_flight, etc.)

```
# Many-to-One: Sentiment Analysis with Keras
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Embedding, SimpleRNN, Dense

model = Sequential([
    Embedding(input_dim=10000,    # vocab size
              output_dim=64,      # embedding dimension
              input_length=200),  # max sequence length

    SimpleRNN(units=128,
              return_sequences=False),  # KEY: False = Many-to-One
                                        # Only h_T is passed forward

    Dense(64, activation='relu'),
    Dense(1, activation='sigmoid')  # binary: positive/negative
])

model.compile(optimizer='adam',
              loss='binary_crossentropy',
              metrics=['accuracy'])

# h_T (the final hidden state) is the 'summary' of the whole review
# This single vector is passed to Dense layers for classification
```

Why the Final Hidden State Works

Each hidden state h_t is computed from x_t AND h_(t-1), meaning h_T has been influenced by every single token in the sequence. It is a fixed-size vector that theoretically encodes the entire sequence history.

Intuition

Think of h_T as a reading comprehension summary.
The RNN reads the entire review word by word, updating its 'understanding'.
After the last word, h_T captures the cumulative sentiment — then we classify from it.
The key limitation: for very long sequences, early tokens have diminishing influence on h_T.

Lecture 6: Other Types of RNN for Gen AI Applications

Beyond Many-to-One, RNNs come in several architectural variants. The choice of architecture is determined by the input-output structure of the task. Understanding each type is essential for designing Gen AI systems.

Overview of All RNN Types

Type	Input | Output | Primary Use Case
One-to-One	Single vector | Single vector | Standard ANN (not truly RNN)
One-to-Many	Single vector | Sequence | Image captioning, music generation from seed
Many-to-One	Sequence | Single vector | Sentiment analysis, classification
Many-to-Many (equal)	Sequence (T) | Sequence (T) | POS tagging, NER, video frame labelling

Type	Input | Output | Primary Use Case
Many-to-Many (unequal)	Sequence (T) | Sequence (T') | Machine translation, summarisation

One-to-Many — Image Captioning

A single input (e.g. a CNN image embedding) is fed as the initial hidden state. The RNN then generates a sequence of tokens one by one, each output fed back as the next input.

```
# One-to-Many: Image Captioning (simplified)
# Step 1: CNN encodes image to feature vector
image_features = cnn_encoder(image)  # shape: (batch, 2048)

# Step 2: Project to RNN hidden size
h_0 = Dense(hidden_size)(image_features)  # initial hidden state

# Step 3: RNN generates tokens autoregressively
# At each step: input = previous token, output = next token probability
generated = []
x = start_token_embedding
h = h_0
for step in range(max_caption_length):
    h = tanh(W_xh @ x + W_hh @ h + b_h)   # update hidden state
    y = softmax(W_hy @ h + b_y)             # predict next token
    token = argmax(y)                        # greedy decoding
    if token == end_token: break
    generated.append(token)
    x = embedding(token)                     # feed back as next input
```

Many-to-Many (Equal) — Sequence Labelling

Each input token produces an output label. The hidden state flows through the sequence, giving each prediction access to left-context (and right-context if bidirectional).

- **Named Entity Recognition (NER):** 'Apple [ORG] CEO Tim [PER] Cook spoke in London [LOC]'
- **Part-of-Speech (POS) tagging:** Each word tagged as NOUN, VERB, ADJ, etc.
- **Frame-level video classification:** Each frame labelled with an action class

Many-to-Many (Unequal) — Encoder-Decoder / Seq2Seq

This is the most important architecture for Gen AI. The input sequence is encoded into a context vector; a separate decoder generates the output sequence from that context. This is the foundation of machine translation, summarisation, and early chatbots.

```
# Seq2Seq (Encoder-Decoder) — Many-to-Many unequal lengths
from tensorflow.keras.layers import LSTM, Dense
from tensorflow.keras.models import Model
from tensorflow.keras import Input

# ENCODER — reads source sequence, outputs context vector
enc_input  = Input(shape=(None, src_vocab_size))
enc_lstm   = LSTM(256, return_state=True)
_, enc_h, enc_c = enc_lstm(enc_input)   # final hidden + cell state
enc_states = [enc_h, enc_c]             # context vector

# DECODER — generates target sequence from context
dec_input  = Input(shape=(None, tgt_vocab_size))
```

```
dec_lstm   = LSTM(256, return_sequences=True, return_state=True)
dec_out, _, _ = dec_lstm(dec_input, initial_state=enc_states)
dec_dense  = Dense(tgt_vocab_size, activation='softmax')
dec_output = dec_dense(dec_out)

model = Model([enc_input, dec_input], dec_output)

# Use case: English -> French translation
# Encoder: reads 'How are you' -> context vector
# Decoder: generates 'Comment allez-vous' token by token
```

Bidirectional RNN — Context from Both Directions

A standard RNN only has left-context at each position. A Bidirectional RNN runs two RNNs — one forward, one backward — and concatenates their hidden states. This gives every token access to full sentence context.

Model	Context Available | Best For
Unidirectional RNN	Only past tokens | Generation tasks (next-token prediction)
Bidirectional RNN	Past + future tokens | NER, sentiment, POS tagging, BERT-style tasks

```
from tensorflow.keras.layers import Bidirectional, LSTM

# Bidirectional wraps any RNN layer
model = Sequential([
    Embedding(vocab_size, 64),
    Bidirectional(LSTM(128)),   # forward + backward, concatenated
    Dense(1, activation='sigmoid')
])
# Output dimension = 128 * 2 = 256 (forward + backward concatenated)
```

Lecture 7: Backpropagation in RNN

Backpropagation in RNNs follows the same chain rule as in standard networks, but with a crucial complication: the same weight matrix W_hh is used at every time step. Gradients must be accumulated across all time steps before updating weights.

Loss Function

For a sequence task, the total loss is the sum of per-step losses (e.g. cross-entropy at each time step):

```
L  =  SUM_t  L_t(y_t, y_hat_t)

L_t  =  -log P(correct token at step t)
```

Gradient Flow — The Chain Rule

For any weight W (e.g. W_hy), the gradient dL/dW is:

```
dL/dW  =  SUM_t  (dL_t / dW)
```

```
dL_t/dW_hy  =  (dL_t/dy_t) * (dy_t/dW_hy)    [simple, one step]

dL_t/dW_hh  =  SUM_(k=1 to t)  (dL_t/dh_t) * PRODUCT_(j=k+1 to t)
                  (dh_j/dh_(j-1)) * (dh_k/dW_hh)
```

Why the product term matters

dh_j / dh_(j-1) = W_hh^T * diag(1 - tanh^2(a_j))

This is the Jacobian of the hidden state with respect to the previous hidden state. It gets multiplied T times across the sequence — this product is what causes vanishing or exploding gradients (covered in Lecture 10).

Parameter Updates

After computing all gradients, parameters are updated by gradient descent:

```
W_xh  <-  W_xh  -  lr * dL/dW_xh
W_hh  <-  W_hh  -  lr * dL/dW_hh
W_hy  <-  W_hy  -  lr * dL/dW_hy
 b_h  <-   b_h  -  lr * dL/db_h
```

Key Difference from Standard Backprop

Aspect	Standard ANN | RNN
Weight sharing	Each layer has unique weights | W_hh is shared across all T time steps
Gradient accumulation	Gradient from one loss | Gradients from T losses summed
Computation graph	Fixed depth | Depth = sequence length T
Gradient path length	Proportional to #layers | Can be T steps deep

Lecture 8: Backpropagating Through Time (BPTT)

Backpropagation Through Time (BPTT) is the algorithm used to train RNNs. It is conceptually identical to standard backpropagation, but applied to the unrolled RNN computation graph — which has depth equal to the sequence length.

The BPTT Algorithm

Step	Action
1. Forward pass	Unroll RNN for T steps; compute h_1, h_2, ..., h_T and outputs y_t; compute losses L_t
2. Initialise gradient	dh_T/dh_T = I (identity)

Step	Action
3. Backward through output	For each t: compute dL_t/dh_t from output layer
4. Backward through time	Propagate gradient backward through h_T -> h_(T-1) -> ... -> h_1
5. Accumulate weight grads	dL/dW_hh += dL_t/dW_hh for each step t
6. Update weights	Apply gradient descent (or Adam, RMSProp, etc.)

Truncated BPTT — Practical Training

For very long sequences (e.g. entire novels, long videos), full BPTT is prohibitively expensive — memory grows linearly with T. Truncated BPTT approximates the full gradient by processing the sequence in chunks.

```
# Truncated BPTT — process long sequence in windows
T       = 10000   # total sequence length
trunc   = 100     # backprop only through last 100 steps

h = np.zeros(hidden_size)   # initial state

for start in range(0, T, trunc):
    chunk = sequence[start : start + trunc]

    # Forward pass over this chunk
    hidden_states = []
    for x_t in chunk:
        h = tanh(W_xh @ x_t + W_hh @ h + b_h)
        hidden_states.append(h)

    # Backward pass — only through this chunk
    # h from previous chunk used as context but NOT differentiated through
    compute_gradients(hidden_states, chunk)
    update_weights()

    # Carry forward hidden state (detached from graph)
    h = h.detach()   # PyTorch: stop gradient flow across chunks
```

BPTT Complexity

Resource	Full BPTT | Truncated BPTT
Memory	O(T) — stores all hidden states | O(trunc) — fixed window
Compute	O(T^2) for gradient accumulation | O(T * trunc) — linear in T
Gradient quality	Exact (limited by precision) | Approximate — loses very long-range deps
Practical use	Short sequences (T < 500) | Long sequences, streaming, online learning

Keras / PyTorch handle BPTT automatically

In Keras, fitting an RNN model uses full BPTT up to the sequence length you provide.
In PyTorch, call loss.backward() and PyTorch traces back through the unrolled graph.
For truncated BPTT in PyTorch: call .detach() on the hidden state between chunks.
The stateful=True flag in Keras LSTM carries state across batches for manual TBPTT.

Lecture 9: Problems with RNN

Despite their theoretical elegance, standard (vanilla) RNNs suffer from several practical problems that severely limit their performance on real-world tasks. Understanding these problems motivated the development of LSTM, GRU, and ultimately the Transformer architecture.

Problem 1 — Vanishing Gradient

When backpropagating through many time steps, gradients are multiplied by W_hh at each step. If the spectral radius of W_hh < 1, gradients shrink exponentially — early time steps receive near-zero gradient updates.

```
Gradient at step k  =  dL/dh_k

                    ~  (W_hh)^(T-k)  *  local_gradient

If ||W_hh|| < 1:  (W_hh)^(T-k) -> 0 as (T-k) grows

=> Early tokens contribute NOTHING to the loss gradient
```

Consequence

The RNN effectively has short-term memory only.
It cannot learn dependencies between tokens that are more than ~10-20 steps apart.
In practice: 'The man who wore a hat and carried an umbrella was ___.' The RNN forgets 'man' by the time it needs to predict the gender agreement of '___'.

Problem 2 — Exploding Gradient

Conversely, if the spectral radius of W_hh > 1, gradients grow exponentially — leading to NaN values and training instability.

Sign of Exploding Gradient	Solution
Loss becomes NaN after a few steps	Gradient clipping: clip gradient norm to max value
Weights update to very large values	Weight initialisation: orthogonal init for W_hh
Training diverges immediately	Reduce learning rate or use adaptive optimiser (Adam)

```
# Gradient Clipping — standard defence against exploding gradients
import torch

optimizer.zero_grad()
loss.backward()

# Clip gradient norm to 1.0 before updating
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=1.0)

optimizer.step()

# In Keras:
optimizer = tf.keras.optimizers.Adam(clipnorm=1.0)
```

Problem 3 — Slow Training on Long Sequences

Because RNNs process tokens sequentially (h_t depends on h_(t-1)), they cannot be parallelised across the time dimension. Training on a sequence of length 1000 requires 1000 sequential matrix multiplications — this cannot be sped up with more GPUs.

Architecture	Parallelisable? | Long-sequence training
RNN / LSTM / GRU	No — sequential dependency | Slow, O(T) sequential steps
1D CNN	Yes — convolution parallelises | Limited receptive field
Transformer	Yes — all tokens processed in parallel | Fast, O(1) depth

Problem 4 — Difficulty with Very Long-Range Dependencies

Even with techniques like LSTM and GRU, capturing dependencies across hundreds or thousands of tokens remains challenging. The hidden state has a fixed size — it must compress an arbitrarily long history into a single vector.

Problem 5 — Fixed-Size Context Bottleneck (Seq2Seq)

In the encoder-decoder architecture, the entire source sequence is compressed into a single context vector h_T. For long sequences, this bottleneck loses information — later tokens overwrite memory of early tokens.

Solution that changed NLP

The Attention Mechanism (Bahdanau, 2015) solved the fixed-context bottleneck.
Instead of one h_T, the decoder attends to ALL encoder hidden states at each decoding step.
This was the direct precursor to the Transformer (2017), which eliminated RNNs entirely.

Summary of RNN Problems

Problem	Cause | Consequence
Vanishing gradient	W_hh^T -> 0 | Cannot learn long-range dependencies
Exploding gradient	W_hh^T -> inf | Training instability, NaN losses
No parallelism	Sequential h_t dependency | Very slow training on long sequences
Context bottleneck	Fixed h_T vector | Information loss in long seq2seq tasks
Short memory	tanh saturation + vanishing gradient | Forgets tokens from > ~20 steps ago

Lecture 10: Vanishing Gradient Problem in Long-Term RNNs

The vanishing gradient problem is the single most important limitation of vanilla RNNs. This lecture examines it in mathematical depth and surveys the architectural solutions that defined the evolution from RNN to LSTM to Transformer.

Mathematical Root Cause

During BPTT, the gradient of the loss at step T with respect to the hidden state at step k is:

```
dL_T / dh_k  =  dL_T/dh_T  *  PRODUCT_(t=k+1 to T)  dh_t/dh_(t-1)

dh_t/dh_(t-1)  =  W_hh^T * diag( 1 - tanh^2(h_(t-1)) )

So:  dL_T/dh_k  ~  (W_hh^T)^(T-k)  *  (tanh derivatives)
```

Two compounding factors both drive the gradient to zero:

- **tanh derivative:** 1 - tanh^2(x) is at most 1.0 and often << 1 when |x| is large (saturated neurons)
- **W_hh spectral radius:** If the largest eigenvalue of W_hh is < 1, repeated multiplication shrinks the product exponentially

Visualising the Gradient Decay

Time Gap (T - k)	Approx Gradient Magnitude | Learning Outcome
1 step	~1.0 | Strong gradient — learns immediately adjacent dependencies
5 steps	~0.1 | Weak but usable — short-range dependencies learned
10 steps	~0.001 | Very weak — marginal learning of medium-range deps
20 steps	~0.000001 | Effectively zero — long-range deps not learned at all
50+ steps	~10^-15 | Numerically zero — model has no gradient signal

Solutions to the Vanishing Gradient Problem

Solution 1 — LSTM (Long Short-Term Memory)

Hochreiter & Schmidhuber (1997). LSTM introduces a cell state — a 'conveyor belt' that carries information across time with minimal modification. Three gates control what to forget, add, and output.

```
Forget gate:   f_t  =  sigmoid( W_f * [h_(t-1), x_t] + b_f )
Input gate:    i_t  =  sigmoid( W_i * [h_(t-1), x_t] + b_i )
Candidate:     g_t  =  tanh(    W_g * [h_(t-1), x_t] + b_g )
    Cell update:   C_t  =  f_t * C_(t-1)  +  i_t * g_t
Output gate:   o_t  =  sigmoid( W_o * [h_(t-1), x_t] + b_o )
         Hidden state:  h_t  =  o_t * tanh( C_t )
```

Why LSTM solves vanishing gradient

The cell state C_t is updated additively: C_t = f_t * C_(t-1) + i_t * g_t
Gradient flows through this additive path with minimal multiplication — no exponential decay.
The forget gate f_t can be set near 1.0 for long-range info that should be remembered.
This 'constant error carousel' (Hochreiter's term) preserves gradients across 100s of steps.

Solution 2 — GRU (Gated Recurrent Unit)

Cho et al. (2014). A simplified LSTM with two gates instead of three. Fewer parameters, similar performance to LSTM, faster training.

```
Reset gate:   r_t  =  sigmoid( W_r * [h_(t-1), x_t] )
Update gate:  z_t  =  sigmoid( W_z * [h_(t-1), x_t] )
```

```
Candidate:    n_t  =  tanh( W_n * [r_t * h_(t-1), x_t] )
Hidden:       h_t  =  (1 - z_t) * h_(t-1)  +  z_t * n_t
```

Model	Gates | Parameters | Performance
Vanilla RNN	0 gates | Fewest | Fails on long sequences
GRU	2 gates (reset, update) | Medium | Close to LSTM, faster
LSTM	3 gates (forget, input, output) + cell | Most | Best on long-range tasks

Solution 3 — Gradient Clipping

The simplest fix for exploding gradients. Rescales the gradient vector if its norm exceeds a threshold. Does NOT fix vanishing — only exploding.

```
# PyTorch gradient clipping
loss.backward()
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
optimizer.step()
```

Solution 4 — The Transformer (The Definitive Solution)

Vaswani et al. (2017) — 'Attention is All You Need'. The Transformer eliminates recurrence entirely. Self-attention creates direct connections between any two tokens in O(1) operations — no gradient path longer than 1 step.

Property	RNN / LSTM | Transformer
Max gradient path length	O(T) — must traverse all T steps | O(1) — direct attention to any position
Parallelism	None — sequential | Full — all positions in parallel
Long-range dependency	Difficult, degrades with T | Effortless, equal for all distances
Memory of sequence	Compressed into fixed h_T | Full attention over all tokens
Training speed	Slow on long sequences | Fast (GPU-parallelisable)

Historical progression

1986: Vanilla RNN — sequential memory, no long-range learning
1997: LSTM — gated cell state, preserves gradients much longer
2014: GRU — streamlined LSTM, faster to train
2015: Attention mechanism — decoder looks at all encoder states
2017: Transformer — self-attention replaces recurrence entirely
2018+: BERT, GPT, T5 — Transformer-based LLMs dominate all NLP

End of Module 19 Notes — Generative AI Course